Hospital Resource Management System

Complete Project Flow — Architecture, Modules, Data & Security

Flutter 3.x · Dart 3.13 | Web + Android | Supabase · PostgreSQL · Auth · Storage · Realtime | Row Level Security multi-tenancy | go_router | ChangeNotifier state

Repository: `e_resource` · version 1.0.0+1 · 37 Dart source files (~5,800 LOC) · 2 HRMS migrations + legacy schema
Generated by static inspection of `lib/` and `supabase/`

Table of Contents

1 What this project is (two systems in one repo)
2 Layered architecture
3 Startup flow — boot to first screen
4 Authentication & session flow
5 Navigation shell & route map
6 Tenancy + RBAC model (the security spine)
7 The per-screen HRMS data-fetch pattern
8 Module flow: Dashboard
9 Module flow: Inventory (items & batches)
10 Module flow: Stock movements (`record_stock_movement`)
11 Module flow: Facilities (departments & warehouses)
12 Module flow: Department requests & review
13 Module flow: Stock transfers (`hrms_transfer_stock`)
14 Module flow: Procurement (suppliers & requisitions)
15 Module flow: Equipment & maintenance
16 Module flow: Users & roles administration
17 Module flow: Reports
18 Role → capability matrix
19 Database schema / entity flow
20 Legacy e-resources subsystem flow
21 Realtime, notifications & favorites
22 End-to-end operational lifecycle
23 Gaps, mismatches & dead code
24 Source file map

1 · What this project is

The repository is mid-transition from an **E-Resources Management System** (document/file sharing portal — package name is still `e_resource`) into a **Hospital Resource Management System (HRMS)** — a multi-tenant, role-gated inventory, procurement and asset platform. Both subsystems coexist in one app.

LEGACY · E-Resources	CURRENT · HRMS	SHARED FOUNDATION	DELIBERATELY ABSENT
Resources + versions	Hospitals / memberships	Supabase Auth	Notifications for HRMS events
Categories, favorites	Departments, warehouses	profiles table	Offline sync
Upload → approve workflow	Items, batches, balances	AppState (ChangeNotifier)	Purchase orders / GRN
Access logs, audit trail	Stock movements, transfers	go_router + AppShell	Patient / clinical records
Notifications, settings	Requests, procurement		Report export
	Assets, maintenance		

> **READ THE README WARNING**
>
> The project explicitly states it is **not production-ready**. It is a working prototype whose security boundary is real (Postgres RLS + `SECURITY DEFINER` functions), but whose operational coverage is partial.

2 · Layered architecture

Layer	Files	Responsibility
Bootstrap	`lib/main.dart`, `lib/core/env.dart`	Load `.env`, initialize Supabase, build AppState, choose between real app and setup screen.
State	`lib/app_state.dart`	Single `ChangeNotifier` holding profile, favorites, settings, unread count; exposes role booleans; owns auth stream + realtime channel. Injected via `AppStateScope` InheritedNotifier.
Routing	`lib/router.dart`, `lib/shell.dart`	GoRouter with a global auth redirect keyed on `refreshListenable: state`; ShellRoute wraps all inner pages in `AppShell`.
Screens	`lib/screens/**`	Stateful widgets. Each HRMS screen owns its own `HrmsService`, hospital selection and `Future` state — no shared view-model.
Services	`lib/services/**`	Thin typed wrappers over Supabase table access + `rpc()` calls. No business rules in Dart — rules live in Postgres.
Backend	`supabase/schema.sql`, `migrations/001–002`	Tables, enums, RLS policies, `SECURITY DEFINER` functions, triggers (versioning, notifications, audit, `updated_at`).

> **KEY ARCHITECTURAL DECISION**
>
> **Authorization is enforced server-side, not in Dart.** Stock balances and movement rows have `REVOKE INSERT, UPDATE, DELETE` from `authenticated`; they can only change through `record_stock_movement()` and `hrms_transfer_stock()`. The client's role checks exist only to show/hide UI — the database is the authority.

3 · Startup flow — boot to first screen

1 · main()

`WidgetsFlutterBinding.ensureInitialized()`; installs a `FlutterError.onError` logger so a startup crash never leaves a blank page.

↓

2 · dotenv.load('.env')

Reads `SUPABASE_URL` and `SUPABASE_ANON_KEY`. `.env` is declared as a Flutter asset in `pubspec.yaml` — a full restart is required after editing it, hot reload will not pick it up. Failure is caught, not fatal.

↓

3 · Env.isConfigured ?

Guard: URL non-empty, anon key non-empty, URL starts with `http`.

↓ *true*

4 · Supabase.initialize(url, publishableKey)

On success `backendReady = true`. Only the anon/publishable key is ever shipped — the service-role key must stay out of client builds.

↓

5 · AppState().init()

Subscribes to `auth.onAuthStateChange`. Restores any persisted Supabase session, which fires `initialSession` → loads profile, favorites, settings, unread count and opens the realtime channel.

↓

6a · backendReady → MaterialApp.router

Light + dark themes, `routerConfig: buildRouter(state)`, initial location `/`.

6b · otherwise → SetupRequiredScreen

Plain `MaterialApp` showing the exact `.env` contents to create plus the instruction to run `supabase/schema.sql` once.

4 · Authentication & session flow

4.1 Sign-in

LoginScreen

Email + password form. Note: there is no register link — self-signup was deliberately removed from the sign-in screen (the router never registers `/register`).

↓

state.signIn(email, password)

AuthService.signIn → auth.signInWithPassword

Supabase Auth issues a session (persisted by the SDK).

↓

ensureProfile()

SELECT from `profiles` where `id = auth.uid()`; if missing, INSERT a row with email + `full_name` from user metadata. This is the "signup safety net" for accounts created before the DB trigger ran.

↓

profile.isActive?

If `false` → `signOut()` then throw `AuthException('This account has been deactivated…')`.

↓

active

Hydrate AppState

In sequence: `refreshProfile()` → `refreshFavorites()` → `refreshSettings()` → `refreshUnread()` → `_subscribeRealtime()` → `notifyListeners()`.

↓

GoRouter.refreshListenable fires the redirect

Redirect → / (HRMS Dashboard)

> **SECOND DEACTIVATION GATE**
>
> `AppState.refreshProfile()` *also* checks `profile.isActive` and force-signs-out. So an account deactivated mid-session is dropped the next time the profile is refreshed via an auth event — but not by a plain screen reload.

4.2 Registration (legacy path, still present in code, not reachable from UI)

register_screen.dart

email, password, full name, department, institution.

↓

auth.signUp(data: {full_name, department, institution})

Creates `auth.users` row.

↓

Trigger on_auth_user_created → handle_new_user()

Inserts the `profiles` row automatically (role defaults to `user`).

↓

ensureProfile()

Fallback manual insert if the trigger has not yet committed.

*A newly registered account has **no HRMS membership**. Every HRMS screen will therefore render "No active hospital membership is assigned to this account." until an administrator grants one (§16). This is the intended onboarding gate.*

4.3 Sign-out

`AppState.signOut()` → `auth.signOut()`, then clears `profile`, `_favoriteIds`, `_unread`, unsubscribes the realtime channel. The auth stream emits `signedOut`, the redirect sends the user to `/login`, and the drawer calls `context.go(Routes.login)`.

5 · Navigation shell & route map

Section	Route	Screen	Status
HRMS rail (visible nav)	`/`	HrmsDashboardScreen	**Live**
	`/inventory`	InventoryScreen	**Live**
	`/stock-movements`	MovementsScreen	**Live**
	`/facilities`	FacilitiesScreen	**Live (hidden from bottom bar)**
	`/requests`	HrmsWorkflowsScreen(requests)	**Live**
	`/transfers`	HrmsWorkflowsScreen(transfers)	**Live**
	`/procurement`	HrmsWorkflowsScreen(procurement)	**Live**
	`/equipment`	HrmsWorkflowsScreen(equipment)	**Live**
	`/maintenance`	HrmsWorkflowsScreen(maintenance)	**Live**
	`/hospital-reports`	HrmsWorkflowsScreen(reports)	**Live**
	`/hospital-users`	HrmsWorkflowsScreen(users)	**Live**
	`/notifications`	NotificationsScreen	**Live (legacy)**
	`/profile`	ProfileScreen	**Live (legacy)**
Orphaned (URL only)	`/login`	LoginScreen	Outside shell (no nav)
	`/resources`, `/resources/:id`, `/resources/:id/edit`	Resources / ResourceDetail / ResourceForm	Routed, not in nav
	`/my-uploads` · `/resources/new`	MyUploads · ResourceForm	Routed, not in nav
	`/favorites`	FavoritesScreen	Routed, not in nav
	`/admin/approvals` · `/admin/categories` · `/admin/users` · `/admin/roles` · `/admin/reports` · `/admin/audit` · `/admin/settings`	7 admin screens	Routed, not in nav
	`/register`	RegisterScreen	**Constant exists, NO route**

5.1 Responsive shell logic

AppShell.build

Reads current location from `GoRouterState.of(context)` and `MediaQuery.sizeOf(context).width`.

↓

width ≥ 900 ?

YES → NavigationRail
extended when ≥ 1200px

Avatar with `initialsOf(name)` + role label at the rail head

13 destinations, notifications badge = unread count

`context.go(destinations[i].route)`

NO → Scaffold + AppBar + Drawer + bottom NavigationBar

AppBar title from `_titleFor(loc)`

Bottom bar = same destinations minus `/facilities`

Drawer adds the **Sign out** action (only place it appears)

Both branches are wrapped by `return AppStateScope(state: state, child: body)`, so any descendant can call `AppStateScope.of(context)`.

6 · Tenancy + RBAC — the security spine

Every HRMS table carries a `hospital_id`. A user's access to a hospital is a row in `hrms_memberships` — and a user may hold **several memberships** (different hospitals, different roles, optionally scoped to a department) at once.

6.1 The single gatekeeper function

```
hrms_has_membership(p_hospital_id uuid, p_roles hrms_membership_role[] DEFAULT NULL) → boolean
LANGUAGE sql STABLE SECURITY DEFINER · SET search_path = public, pg_temp

exists (select 1 from hrms_memberships m
  where m.user_id = auth.uid() and m.hospital_id = p_hospital_id
    and m.is_active and (p_roles is null or m.role = any(p_roles)))
```

Runs as the function owner, so a user can test membership without SELECT rights on the membership table itself. `REVOKE ALL FROM public` + `GRANT EXECUTE TO authenticated` — anon cannot call it. The `search_path` pinning blocks search-path hijacking.

6.2 The ten membership roles

Role	Domain of authority
`hospital_admin`	Everything inside one hospital: facilities, items, stock, reviews, transfers, procurement, assets, membership grants.
`inventory_manager`	Item & batch catalog, all movement kinds, transfers, request review.
`storekeeper`	Issue / return / transfer legs, request review; **cannot** receipt, adjust or write off stock.
`pharmacist`	Medicine catalog + batches, receipt / issue / adjustment / wastage, request review.
`procurement_officer`	Suppliers, requisition lifecycle, request review.
`department_head`	Submit & review department requests; requisitions.
`healthcare_worker`	Submit requests, report maintenance (read-only elsewhere).
`biomedical_engineer`	Medical assets and maintenance work orders.
`finance_officer`	Requisition status changes; dashboard in a cost-oriented view.
`auditor`	Read-only oversight; dashboard framed as "Audit activity".

6.3 Defence in depth — three independent layers

Layer 1 · Dart UI gating

Screens compute `_roles` from the selected hospital and hide buttons — e.g. `_canCreate` in `workflows_screen.dart:116`, `_canManageInventory`/`_canMoveStock` in the dashboard.

↓

bypassing this gains nothing

Layer 2 · PostgreSQL privileges

`REVOKE INSERT, UPDATE, DELETE ON stock_balances, stock_movements FROM anon, authenticated;` `stock_transfers` is grant-limited to `SELECT, INSERT` only. Direct tampering is a permission error, not a policy check.

↓

Layer 3 · RLS policies per table

Every table has `ENABLE ROW LEVEL SECURITY` plus a "`… read …`" policy using `hrms_has_membership(hospital_id)` and a write policy restricted to an explicit role array — with both `USING` and `WITH CHECK`, so a row cannot be re-parented into another hospital.

6.4 Write-path summary

Table	Read	Write
`hospitals`	Any active member	**No policy at all → read-only for all clients**
`departments` · `warehouses`	Any member	`hospital_admin` only (`FOR ALL`)
`hrms_memberships`	Own rows, or `hospital_admin`	Via `SECURITY DEFINER` RPCs only
`inventory_items` · `inventory_batches`	Any member	`hospital_admin`, `inventory_manager`, `pharmacist`
`stock_balances`	Any member	**Function-only (privileges revoked)**
`stock_movements`	Any member	**Function-only (privileges revoked)**
`resource_requests`	Any member	INSERT: self only (`requested_by = auth.uid()`); UPDATE: admin / inventory_manager / department_head / storekeeper / pharmacist
`resource_request_lines`	Any member	INSERT: only into **your own pending** request; UPDATE: reviewers
`stock_transfers`	Any member	INSERT: admin / inventory_manager / storekeeper, `actor_id = auth.uid()`; **no UPDATE or DELETE**
`hrms_suppliers`	Any member	`FOR ALL`: admin, procurement_officer
`purchase_requisitions`	Any member	INSERT: self; UPDATE: admin, procurement_officer, finance_officer
`medical_assets`	Any member	`FOR ALL`: admin, biomedical_engineer
`maintenance_work_orders`	Any member	INSERT: self; UPDATE: admin, biomedical_engineer

7 · The per-screen HRMS data-fetch pattern

All six HRMS screens repeat the same shape. Understanding it once explains the whole module set.

initState()

Instantiate a private `HrmsService`; set `_hospitals = _service.hospitals()` as a `late Future`.

↓

FutureBuilder<List<HrmsHospital>>

States: error → error panel (+ Retry); no data → spinner; empty list → "No active hospital membership is assigned to this account."

↓

_hospitalId == null ?

Auto-select inside `WidgetsBinding.instance.addPostFrameCallback` → `_select(hospitals.first)`. Selecting during build is illegal in Flutter, hence the post-frame deferral.

↓

_select(hospital)

Sets `_hospitalId`, caches `_roles = hospital.roles` (drives every button), and starts `_rows = _load(hospitalId)` — which first fetches the picker dependencies (departments / warehouses / items / suppliers / assets), then the module list.

↓

Hospital dropdown + list + action buttons

Switching hospital re-runs `_select`. After a mutation, screens call `setState(() => _rows = _load(_hospitalId!))` — a full re-query, not an optimistic patch.

7.1 Service call shape

`HrmsService` uses only two Supabase verbs: query-builder chains (`.from(t).select(cols).eq('hospital_id', id).order().limit(n)`) and `.rpc(fn, params: {p_…})` for the privileged operations. Every list read carries an explicit `hospital_id` filter *and* a hard `limit` (100 / 200 / 500) — pagination is not implemented.

> **CONSEQUENCE OF THE PER-SCREEN PATTERN**
>
> The selected hospital lives in each screen's private state and is **not persisted or shared**. Navigating from Inventory to Transfers re-fetches the hospital list and silently resets to `hospitals.first`. A user with three hospitals must re-pick the tenant on every page. There is no `AppState.currentHospitalId`.

8 · Module flow: Dashboard

`HrmsService.dashboard(hospitalId)` runs four queries concurrently via `Future.wait`:

#	Query	Becomes
1	`inventory(hospitalId)`	All items + aggregated quantities → KPI + `lowStockItems` + `stockedItemsCount`
2	`movements(hospitalId)`	Recent 100 stock movements
3	`inventory_batches` where `expires_on` between today and today+30d, not quarantined	`expiringBatchCount`

4	`inventory_batches` where `expires_on` < today, not quarantined	`expiredBatchCount`

The dashboard then renders a **role-adaptive panel**: the same data, different framing and shortcuts.

Roles held	Effect
`hospital_admin`	Administration shortcuts; full KPI set
`pharmacist` / `inventory_manager` / `hospital_admin`	Expiry and batch panels surfaced
`procurement_officer`	Low-stock heading re-labelled as a reorder prompt; procurement shortcut
`auditor` / `finance_officer`	Activity feed labelled "Audit activity"; oversight shortcut
`department_head` / `healthcare_worker`	Request-submission shortcut
`biomedical_engineer`	Equipment / maintenance shortcut
none of the above	Neutral summary text only

Guards used: `_canManageInventory` (admin / inventory_manager / pharmacist) and `_canMoveStock` (adds storekeeper).

9 · Module flow: Inventory (items & batches)

9.1 Reading the stock view

A · inventory_items

`.eq('hospital_id', h).eq('is_active', true)` · optional `.or('name.ilike.%term%,item_code.ilike.%term%')` · `order('name').limit(500)`

↓ then, in parallel intent ↓

B · stock_balances

`select('item_id,quantity').eq('hospital_id', h)` (+ `.eq('warehouse_id', w)` when a warehouse filter is active)

↓

C · Client-side aggregation

Build `Map<itemId, double>` by summing balances, then `InventoryItem.fromMap(row, quantities[row['id']] ?? 0)`. Items with no balance row are shown as **0 on hand** rather than filtered out.

↓

D · Row rendering

Code, name, kind (`supply/medicine`), unit, quantity, and a low-stock chip when `quantity <= minimum_stock`.

9.2 Create item

`item_code` (unique per hospital), name, kind enum, unit, `minimum_stock`, `reorder_level`; `created_by = auth.uid()`. Allowed for admin / inventory_manager / pharmacist.

9.3 Batch create + receipt chain (the medicine path)

Item kind == 'medicine' and no matching batch?

↓

Batch dialog first

batch_number, expires_on, manufactured_on, unit_cost. Uniqueness `(item_id, batch_number)` — one batch record per item, reused by later receipts.

↓

_createBatch returns bool; flow aborts if false

createBatch() → inventory_batches INSERT

↓

Then open the movement dialog

Batches are listed ordered by `expires_on` with `is_quarantined = false`, i.e. earliest-expiry (FEFO) candidates first.

↓

recordMovement(...batchId) → RPC

See §10 for the server-side transaction.

9.4 Movement dialog — kind options and their server-side meaning

UI option	`kind`	Effect on balance	Roles accepted by the function
Receive stock	`receipt`	**+ quantity**	admin, inventory_manager, pharmacist
Issue stock	`issue`	**− quantity**	admin, inventory_manager, storekeeper, pharmacist
Positive adjustment	`adjustment`	**+ quantity**	admin, inventory_manager, pharmacist
Record wastage	`wastage`	**− quantity**	admin, inventory_manager, pharmacist
Return	`return`	**+ quantity**	admin, inventory_manager, storekeeper, pharmacist
(transfer legs — function-generated only)	`transfer_out` / `transfer_in`	− / +	Written exclusively by `hrms_transfer_stock`

The dialog pre-selects `receipt` for catalog managers and `issue` otherwise; quantity is a user-entered positive number in the item's own unit.

10 · Module flow: Stock movements — `record_stock_movement`

The only legal way to change a balance. `plpgsql`, `SECURITY DEFINER`, returns the new movement id.

1 · Auth guard

`IF auth.uid() IS NULL → RAISE 'Authentication required'`

↓

2 · Quantity guard

`NULL or ≤ 0 → 'Quantity must be greater than zero'`. Column-level `CHECK (quantity > 0)` backs this up.

↓

3 · Role gate (kind-dependent)

`receipt/adjustment/wastage → {hospital_admin, inventory_manager, pharmacist}`; everything else → that set **plus** `storekeeper`. Failure → 'Not authorized for this hospital operation'.

↓

4 · Batch eligibility + row lock

If `p_batch_id` given: `SELECT … FOR UPDATE` requiring, for `issue`, that `NOT is_quarantined` and `expires_on IS NULL OR expires_on ≥ current_date`. Not found → 'Batch not found or unavailable for issue'.
If **no** batch and the item's kind is `medicine` → 'Medicine movements require a batch'. **Expired and quarantined medicine can therefore never leave stock.**

↓

5 · Ensure balance row exists

INSERT with `quantity = 0` · `ON CONFLICT (warehouse_id, item_id, batch_id) DO NOTHING`. Uses the `UNIQUE NULLS NOT DISTINCT` index so a NULL batch still conflicts correctly.

↓

6 · Lock the balance

`SELECT quantity … FOR UPDATE` — serialises concurrent issues for the same (warehouse, item, batch).

↓

7 · Overspend guard

For `issue` / `wastage` / `transfer_out`: `v_balance < p_quantity → 'Insufficient stock'` and the whole transaction rolls back.

↓

8 · Apply delta

Single `UPDATE` with a `CASE`: subtract for the three outbound kinds, add otherwise; bump `updated_at`. `CHECK (quantity ≥ 0)` is a final structural guarantee.

↓

9 · Append immutable history

INSERT into `stock_movements` with `actor_id = auth.uid()`, `reason`, `reference`; `RETURNING id` → the function's result.

> **Design property:** steps 5–9 are one transaction, so `stock_balances` is always derivable-consistent with `stock_movements`. Movements are append-only: no client can update or delete a history row, making the ledger

> an audit trail in itself.

11 · Module flow: Facilities

FacilitiesScreen

Hospital picked as in §7; only `hospital_admin` sees create/toggle controls.

↓

Departments

create: `hospital_id` + name + optional code. Constraints `UNIQUE(hospital_id, name)` and `UNIQUE(hospital_id, code)`. Activate/deactivate via `setDepartmentActive`.

↓

Warehouses

create: hospital + optional department + name + location; `setWarehouseActive` for the inverse. `warehouses(hospitalId, includeInactive: true)` when administering, default active-only when picking a transfer endpoint.

WHY `UNIQUE(ID, HOSPITAL_ID)` IS EVERYWHERE

Composite uniques on `(id, hospital_id)` let child tables declare `FOREIGN KEY (x_id, hospital_id) REFERENCES parent(id, hospital_id)`. This makes **cross-tenant references structurally impossible**: you cannot attach a warehouse from hospital A to a balance in hospital B, even with a crafted request.

12 · Module flow: Department requests & review

REQUESTER submitRequest	REVIEWER reviewRequest(id, status, note)	LISTING `requests(hospitalId)`
1 · INSERT `resource_requests` department, purpose, priority, required_by, `requested_by = uid`, status defaults `'pending'`	UPDATE `resource_requests` SET status, `reviewed_by = uid`, `review_note`	Embeds `departments(name)` and `resource_request_lines(id, quantity, approved_quantity, inventory_items(name, item_code, unit))`
2 · `select('id').single()`	UI offers **Approve** → `'approved'` UI offers **Reject** → `'rejected'`	order by created_at desc · limit 200
3 · INSERT `resource_request_lines` item + quantity	RLS roles: admin, inventory_manager, department_head, storekeeper, pharmacist	After any action the screen re-queries the whole list
RLS: INSERT allowed when member AND `requested_by = auth.uid()`	Enum also defines `fulfilled`, `cancelled` — not reachable in UI	

TWO OPEN LOOPS IN THIS FLOW

- `approved_quantity` exists on the line and is readable, but **no UI writes it** — approval is a blanket yes/no on the whole request, not a partial allocation.
- **Approval does not move stock.** Nothing links an approved request to an `issue` movement; a storekeeper must go to Inventory and record the issue by hand. There is no "fulfil" action and no reference linking movement → request id (though `reference` is free-text and could carry it).

13 · Module flow: Stock transfers — `hrms_transfer_stock`

1 · Guards

Authenticated · quantity > 0 · `source ≠ destination` ('Choose two different warehouses', backed by `CHECK`) · role ∈ {`hospital_admin, inventory_manager, storekeeper`}.

↓

2 · Batch check (stricter than §10)

When a batch is supplied the quarantine + expiry test is applied **unconditionally** — transfers of expired or quarantined stock are always refused. Medicine without a batch → 'Medicine transfers require a valid batch'.

↓

3 · Lock + verify source

`SELECT … FOR UPDATE` on the source balance; `coalesce(v_qty,0) < p_quantity → 'Insufficient stock at source warehouse'`.

↓

4 · Debit source

`quantity = quantity - p_quantity`, bump `updated_at`.

↓

5 · Ensure destination row

`INSERT 0 + ON CONFLICT DO NOTHING` — needed because the destination balance may not exist yet.

↓

6 · Credit destination

`quantity = quantity + p_quantity`.

↓

7 · Write BOTH ledger legs

Two-row INSERT into `stock_movements`: `transfer_out` at source and `transfer_in` at destination, same quantity, reason, reference and `actor_id`. Stock is conserved by construction.

↓

8 · Record the transfer header

INSERT `stock_transfers` (status defaults `'completed'`) `RETURNING id`.

Client listing embeds the two warehouse names through explicit FK disambiguation — `source:warehouses!stock_transfers_source_warehouse_id_fkey(name)` and `destination:warehouses!stock_transfers_destination_warehouse_id_fkey(name)` — because a single table holds two FKs to `warehouses`.

> **Irreversible by design:** `stock_transfers` is granted only `SELECT, INSERT`. The `'cancelled'` enum value exists but no policy permits UPDATE, so a mistaken transfer can only be corrected by a compensating reverse transfer.

14 · Module flow: Procurement

Suppliers

"Add supplier" icon appears only for `hospital_admin/procurement_officer`. Fields: name (unique per hospital), contact, email, phone, address, `created_by`. RLS `FOR ALL` for those two roles.

↓

Requisition created

`createRequisition` → title, justification, optional department / supplier / estimated cost / required-by, hard-coded `status: 'submitted'`, `created_by = uid`. Any active member may submit.

↓

Review

Popup → **Approve** (`'approved'`) or **Reject** (`'rejected'`). UPDATE policy: `hospital_admin`, `procurement_officer`, `finance_officer`.

↓

dead end

`'received'` / `'draft'` / `'cancelled'`

Defined in `hrms_procurement_status` but not offered by the UI, and there is no purchase order, goods-receipt note, or stock receipt linked to a requisition. The procurement flow ends at approval — stock is then received manually in Inventory.

A natural reorder trigger — `reorder_level` on the item, or an approved request — is not wired to requisition creation.

15 · Module flow: Equipment & maintenance

15.1 Medical assets

`createAsset`: `asset_tag` (unique per hospital), name, department, serial, manufacturer, model, location, `purchased_on`, `warranty_until`, notes, `created_by`. Status enum defaults `'available'`: `available · assigned · under_maintenance · out_of_service · quarantined · decommissioned`. RLS write: `hospital_admin`, `biomedical_engineer`.

> **UI / RLS MISMATCH — CONFIRMED**
>
> `workflows_screen.dart:136` grants `_canCreate` for the equipment and maintenance modules to `{hospital_admin, biomedical_engineer, healthcare_worker}`. But the database allows asset writes only for `{hospital_admin, biomedical_engineer}`. A `healthcare_worker` is therefore shown an "Add equipment" button that **always fails on the server** with 'new row violates row-level security policy'. For *maintenance* the opposite is true — every member may report — so the shared `||` branch is too narrow there as well. The two modules need separate permission sets.

15.2 Maintenance work orders

Report

`createMaintenance(assetId, title, description, priority, scheduled_for)`, `reported_by = uid`, status defaults `'open'`. RLS: any member, self-atttributed.

↓

open → in_progress

`setMaintenanceStatus(id, 'in_progress')` — allowed for admin / biomedical_engineer.

↓

→ completed (+ resolution) | cancelled

Same call writes `resolution` text.

*Not implemented: `assigned_to` is a column but no UI sets it; asset status is **not** changed when a work order opens or closes, so `under_maintenance` must be maintained by hand and can silently drift from the work-order table. No scheduled/service-interval engine exists.*

16 · Module flow: Users & roles administration

Whole module is `hospital_admin`-only in UI, and every one of the three RPCs re-checks that server-side.

List — `hrms_list_hospital_users(p_hospital_id)`

Joins `hrms_memberships` → `auth.users` to expose email (normally unreachable to clients). The whole thing is gated: the `WHERE` includes `hrms_has_membership(p_hospital_id, '{hospital_admin}')`, so a non-admin gets an **empty result**, not an error.

↓

Add — `hrms_add_member(hospital, email, role, dept)`

1) admin check → 'Only a hospital administrator can assign memberships'. 2) `SELECT id FROM auth.users WHERE lower(email) = lower(trim(p_email))`; if none → **'No registered account found for this email'**. 3) Look for an identical `(user, hospital, department, role)` membership; if found just flip `is_active = true`, else INSERT. Idempotent — re-adding cannot duplicate a role.

↓

Deactivate — `hrms_set_membership_active(id, bool)`

Re-reads the membership's hospital, re-checks admin, then refuses self-deactivation → **'You cannot deactivate your own membership'** (anti-lockout). Sets the flag only; never deletes.

DELIBERATE ONBOARDING ORDER

Public self-registration was removed and membership can only be granted to an **already-existing account**. So the operational flow is: user signs up through a Supabase-console-created account (or a retained legacy path) → an administrator attaches them to a hospital with a role → HRMS screens unlock. There is no bootstrap mechanism for the *first* `hospital_admin`; README assigns that to "a trusted administrative process" (direct SQL).

Note the granularity: memberships are per (hospital, department, role). A user can be `pharmacist` in one hospital and `auditor` in another, and `HrmsService.hospitals()` collapses this into `hospital → Set<roles>` — which is exactly what the UI consumes. Department-level role separation is flattened away at read time.

17 · Module flow: Reports

`HrmsModule.reports` is not a separate service: `_load()` calls `dashboard(id)` and reshapes it into a flat label/value list with section markers (`{'_section': …}`).

Section	Rows
Inventory summary	Catalog items · Items currently stocked · Low or out of stock · Batches expiring within 30 days · Expired batches
Low stock items	Per item: "`qty unit on hand · minimum N`"
Recent stock activity	The raw recent-movement maps (max 100)

`_canCreate` is `false` for reports — read-only, no hospital-member role restriction on viewing. No CSV/PDF export, no date range, no per-department breakdown, no valuation (`unit_cost` is stored on batches but never aggregated into a report).

18 · Role → capability matrix

Source: RLS policies + SECURITY DEFINER role arrays (authoritative). ✔ permitted, ✘ refused.

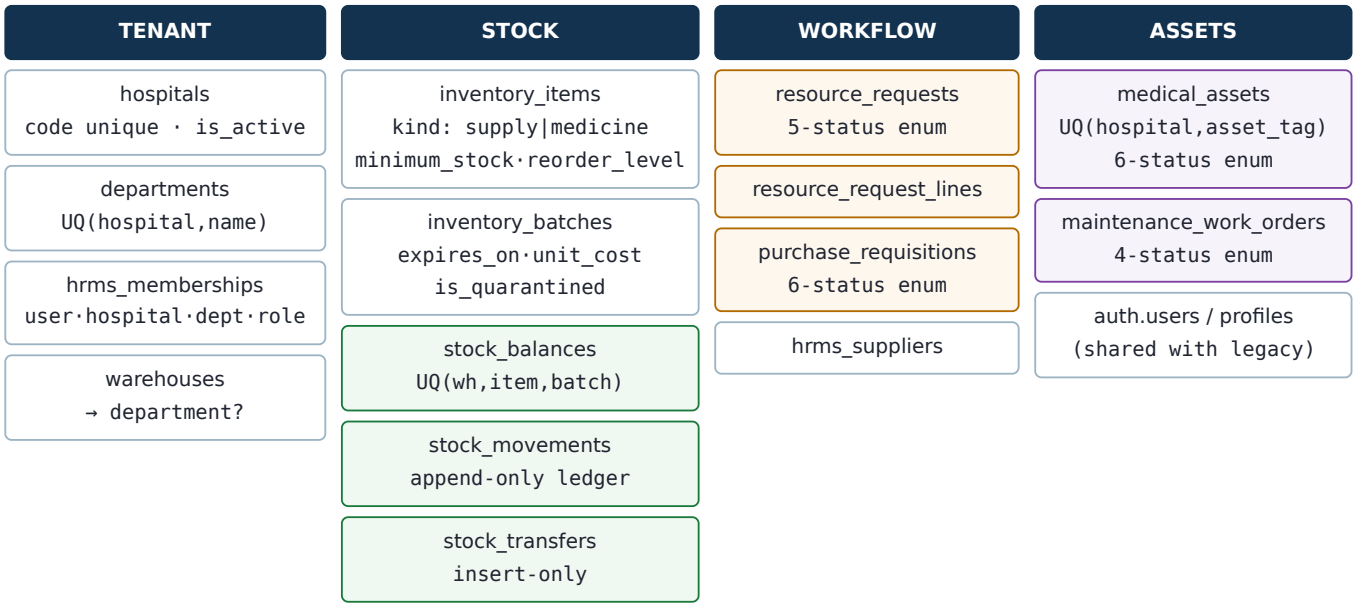
Capability (server-side)	hospital admin	inventory manager	store keeper	pharmacist	procurement	dept head	health worker	biomed eng.	finance	auditor
Read hospital data	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔
Departments / warehouses	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘
Items & batches (catalog)	✔	✔	✘	✔	✘	✘	✘	✘	✘	✘
Receipt / adjustment / wastage	✔	✔	✘	✔	✘	✘	✘	✘	✘	✘
Issue / return	✔	✔	✔	✔	✘	✘	✘	✘	✘	✘
Stock transfer	✔	✔	✔	✘	✘	✘	✘	✘	✘	✘
Submit request	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔
Review request	✔	✔	✔	✔	✘	✔	✘	✘	✘	✘
Suppliers	✔	✘	✘	✘	✔	✘	✘	✘	✘	✘
Submit requisition	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔
Requisition status	✔	✘	✘	✘	✔	✘	✘	✘	✔	✘
Medical assets	✔	✘	✘	✘	✘	✘	✘	✔	✘	✘
Report maintenance	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔

Close work order	✔	✘	✘	✘	✘	✘	✘	✔	✘	✘
Manage memberships	✔	✘	✘	✘	✘	✘	✘	✘	✘	✘

TWO STRUCTURAL OBSERVATIONS

- **No read-only role.** `auditor` and `finance_officer` are treated as ordinary members for reads, but "member" is `p_roles IS NULL` — i.e. *any* role, including `healthcare_worker`, can read every hospital record: balances, costs, suppliers, requisitions. There is no least-privilege read tier.
- **UI gating is narrower than the DB in several places** (e.g. `finance_officer` may change requisition status but `_canCreate` does not list them for procurement) — so some permitted actions are unreachable without editing the Dart. The inverse also occurs (§15).

19 · Database schema / entity flow



19.1 Enum / state inventory

Type	Values	Default & reachable from UI
`hrms_membership_role`	10 roles (§6.2)	set by admin RPC
`hrms_item_kind`	`supply`, `medicine`	`supply`; both selectable
`hrms_movement_kind`	receipt, issue, adjustment, transfer_in, transfer_out, return, wastage	5 selectable; 2 transfer legs function-only
`hrms_request_status`	pending, approved, rejected, `fulfilled`, `cancelled`	pending → approve/reject; last two unreachable
`hrms_transfer_status`	completed, `cancelled`	always completed; no UPDATE grant
`hrms_procurement_status`	`draft`, submitted, approved, rejected, `received`, `cancelled`	client forces submitted; approve/reject only
`hrms_asset_status`	available, assigned, under_maintenance, out_of_service, quarantined, decommissioned	available; **no UI control at all**
`hrms_maintenance_status`	open, in_progress, completed, cancelled	open → all four reachable
priority (both tables)	low / normal / high / urgent	CHECK constraint, default normal

Money/quantity types: `numeric(14,3)` for quantities, `numeric(14,2)` for costs — deliberately not `float`, so balance arithmetic is exact. Dart models all quantities as `double`, which is safe at these magnitudes but is the one place exactness is lost.

20 · Legacy e-resources subsystem flow

Defined in `supabase/schema.sql` + `lib/services/resource_service.dart` / `admin_service.dart`. Still fully coded; reachable only by typing a URL (§5).

20.1 Upload → publish

ResourceFormScreen

`file_picker`; extension whitelist + 100 MB cap per `FileRules`; type inferred from extension; categories multi-select.

↓

Storage upload

Bucket `resources`, path `uploads/{uid}/{timestamp}.{ext}`, `upsert: false`. Storage policy requires an authenticated uploader.

↓

INSERT resources

Metadata + `storage_path/file_size/mime_type`, `uploaded_by = uid`, status **`pending`** — or `approved` immediately when the uploader `canApprove` (admin/librarian).

↓

resource_categories rows

Delete-then-insert of the join rows.

↓

DB triggers fire automatically

notify_approvers_on_upload

Inserts notification rows for approver roles → `pending` resources reach reviewers without any HRMS/notification code in Dart.

20.2 Approval, versioning, consumption

Flow	Mechanics
Approval	`setStatus(id, approved|rejected|archived)` + `setAvailability(active|inactive|restricted|archived)`. Trigger `notify_on_resource_change` informs the uploader.
Versioning	`addVersion()` uploads to `versions/{id}/{n}.{ext}`, inserts `resource_versions`, bumps `resources.version`. Trigger `resource_version_snapshot` snapshots the previous state on update. `restoreVersion()` snapshots current, then repoints the resource (version advances, never rewinds).
View / download	`log_access` RPC writes `access_logs`, then `createSignedUrl(path, 300)` — a 5-minute URL. `isDownloadable` requires a file, `approved` status, and `active/restricted` availability.
Audit	Trigger `audit_resource_changes` writes `audit_logs` (readable only by `admin/librarian`); `AuditScreen` and `accessLogs()` surface both.
Discovery	`ResourceQuery`: free-text `or()` over title/description/author, type, category, status (defaults to `approved` only), favorites, mine.
Analytics	`counts()`, `popularByDownloads()`, `downloadsPerDay(days: 14)` → `fl_chart` in `ReportsScreen`; `broadcast()` sends a system notification to all users.
Roles	Independent 4-value ladder: `user → staff → librarian → admin`, via `profiles.role` + helper SQL functions `current_role() / is_admin() / is_staff_level() /`

	`can_manage_resources()`. **Not connected to HRMS memberships.**

> **TWO AUTHORIZATION MODELS COEXIST**
>
> Legacy RLS decides on `profiles.role` (global). HRMS RLS decides on `hrms_memberships.role` (per hospital). A user can simultaneously be a legacy `user` and an HRMS `hospital_admin`, or a legacy `admin` with no hospital access. `AppState.isAdmin / canApprove / canUpload` read only the legacy field, while every HRMS screen reads only the membership set — so the sidebar, the profile badge and the HRMS buttons can legitimately disagree about who a user is. Unifying these is the largest remaining design task.

21 · Realtime, notifications & favorites

Channel `'public:notifications'`

Opened on sign-in only. `onPostgresChanges(insert, table: notifications, filter: user_id = <uid>)`.

↓

Callback → AppState

`_unread += 1; notifyListeners()` → shell badge updates live and `GoRouter` refresh runs.

↓

NotificationsScreen

`list(limit: 100)`, `markRead(id)`, `markAllRead()`. RLS lets a user read and update only their own rows.

Favorites are legacy-only: `AppState.refreshFavorites()` loads the id set at sign-in, `toggleFavorite()` upserts/deletes and patches the in-memory set. `FavoritesScreen` is routed but not in the nav.

> **HRMS emits no notifications.** An approved request, an overdue work order, a low-stock item or an expiring batch never produces a notification row — the realtime pipeline is wired only to the legacy `resources` triggers. README lists this as unimplemented.

22 · End-to-end operational lifecycle

The complete real-world loop the system supports, with the module responsible at each stage and the gaps marked.

#	Stage	Actor / role	Mechanism	State change
1	Provision tenant	**Trusted SQL / service role**	INSERT `hospitals`	Hospital exists; no client can do this
2	Structure	`hospital_admin`	Facilities module	Departments + warehouses active
3	Staff	`hospital_admin`	`hrms_add_member`	Membership → roles resolve in UI
4	Catalogue	`inventory_manager` / `pharmacist`	Inventory module	Items (+ batches for medicine)
5	Receive	same	`record_stock_movement(receipt)`	Balance ↑ · movement row
6	Signal need	`healthcare_worker` / `department_head`	Requests module	Request `pending`
7	Decide	reviewer roles	`reviewRequest`	`approved`/`rejected` ← does not move stock
8	Allocate	`storekeeper`	Inventory → `issue`	Balance ↓ · FEFO batch enforced
9	Reposition	admin / inv. mgr / storekeeper	`hrms_transfer_stock`	Both balances + 2 legs + header
10	Reorder	`procurement_officer`	Suppliers + requisition	`submitted` → `approved` ← no PO, no GRN
11	Restock	`storekeeper`	Manual receipt, referencing the requisition by free text	Balance ↑ ← `unlinked`
12	Control losses	inv. mgr / pharmacist	`adjustment` / `wastage` / `return`	Balance ± · reason recorded
13	Manage equipment	`biomedical_engineer`	Assets + work orders	Asset register · `open` → `completed` ← asset status manual
14	Oversee	`auditor` / `finance_officer`	Reports module + movements ledger	Read-only view ← no export

23 · Gaps, mismatches & dead code

#	Finding	Evidence	Impact
1	Equipment/maintenance UI gate ≠ DB policy	`workflows_screen.dart:136` vs 002 migration policies	**Healthcare workers see a button that always fails; biomed-only writes for reporting are over-restricted. Split the two branches.**
2	Request approval does not issue stock	No caller links `record_stock_movement` to a request	Approval and allocation are untracked, separate acts; no fulfilment evidence.
3	`approved_quantity` never written	column exists; `reviewRequest` sets status only	No partial approval — an over-optimistic "approve all or nothing".

4	Asset status never updated	`medical_assets.status` has no UI writer	Register drifts from work-order reality; `under_maintenance` effectively unused.
5	Hospital selection is per-screen and ephemeral	each screen holds private `_hospitalId`, auto-picks `.first`	**Multi-hospital users re-choose the tenant on every page; risk of acting in the wrong hospital.**
6	No least-privilege read tier	`p_roles IS NULL` = any membership	**Every member reads costs, suppliers and requisitions; HR-sensitive data over-exposed.**
7	Two disconnected role models	`profiles.role` vs `hrms_memberships.role`	Inconsistent capabilities and confusing profile screen.
8	No bootstrap admin	`hrms_add_member` requires an existing admin	First-admin provisioning must be manual SQL — blocks new tenants.
9	HRMS emits zero notifications	realtime + triggers cover `notifications` from legacy tables only	**No expiry / low-stock / approval alerts; README confirms unimplemented.**
10	Unreachable status enum values	`fulfilled`, `cancelled`, `draft`, `received`, transfer `cancelled`	Data model is ahead of the workflow; cancelled transfers are impossible by grant.
11	Legacy admin/resource screens routed but not in nav	`shell.dart:164` spreads only `_userDestinations`	`_staffDestinations` and `_adminDestinations` are dead constants; 9 legacy pages are URL-only.
12	`/register` route never registered	`constants.dart` defines it; `router.dart` omits it	`register_screen.dart` is unreachable; signup only via Supabase console.
13	No pagination	hard `limit(500/200/100)` everywhere	Silently truncated lists at real hospital scale; low-stock counts under-report.
14	Aggregation done client-side	`inventory()` sums balances in Dart	Two round-trips and O(rows) transfer; a SQL view / RPC would be exact and cheaper.
15	No offline sync, no report export, no audit/report layer for HRMS	README scope notes	Blocks field use on unstable hospital networks.
16	Client `updated_at` not always written	several INSERT/UPDATE paths omit it (tables lack a `touch` trigger, unlike legacy)	`updated_at` can be stale on HRMS rows; don't build deltas on it.
17	`.env` bundled as a Flutter asset	`pubspec.yaml: assets: - .env`	Ship-anonymous key only. Any secret placed in `.env` is readable from the web bundle — verify before production.
18	Batch expiry check skips transfers of unbatched supplies	002 §transfer has no date branch for `p_batch_id IS NULL`	Correct today; revisit if non-medicine batches are introduced.

24 · Source file map

Path	Lines	Role in the flow
`lib/main.dart`	154	Bootstrap + SetupRequiredScreen fallback
`lib/app_state.dart`	185	Session/profile/favorites/unread + realtime + legacy role flags
`lib/router.dart`	165	25 routes + global auth redirect
`lib/shell.dart`	335	Rail / drawer + bottom bar; AppStateScope injection
`lib/models/models.dart`	479	Legacy enums + Profile/Resource/Category/Notification/Audit models
`lib/core/constants.dart`	58	Route table + file-upload rules
`lib/core/env.dart`	13	Config guard
`lib/services/hrms_service.dart`	601	All HRMS reads + RPC wrappers + HRMS view models
`lib/services/resource_service.dart`	329	Legacy CRUD, storage, versions, favorites, signed URLs
`lib/services/admin_service.dart`	271	Legacy users/roles/categories/settings/audit + NotificationService
`lib/services/auth_service.dart`	100	signIn/register/ensureProfile/profile edits
`lib/screens/hrms/dashboard_screen.dart`	620	KPIs, role-adaptive panel
`lib/screens/hrms/inventory_screen.dart`	819	Items, batches, movement dialog
`lib/screens/hrms/workflows_screen.dart`	793	Seven modules through one `HrmsModule` switch
`lib/screens/hrms/facilities_screen.dart`	404	Departments + warehouses
`lib/screens/hrms/movements_screen.dart`	148	Read-only movement ledger
`lib/screens/hrms/hrms_form_widgets.dart`	130	Shared form scaffolding
`lib/screens/auth/` · `lib/screens/admin/` · `lib/screens/*.dart`	~1,900	Login + 7 legacy admin + 7 legacy resource screens
`supabase/schema.sql`	454	Legacy 11 tables, 20+ policies, 9 triggers/functions
`supabase/migrations/001_hrms_inventory_foundation.sql`	248	Tenant + stock core, `hrms_has_membership`, `record_stock_movement`
`supabase/migrations/002_hrms_workflows.sql`	277	7 workflow tables, `hrms_transfer_stock`, membership RPCs

Hospital Resource Management System — Complete Project Flow. Produced by static inspection of the repository at branch `main`. File and policy references cite the exact source; enum/status behaviour reflects the combination of grants, RLS policies and RPC role arrays in `supabase/migrations/`, which take precedence over Dart UI gating.